Estimations of Quantitative Features for SignBase Objects

23/02/2024

Description

This file contains the code for estimating the feature values (TTR, unigram entropy, entropy rate, repetition rate) per sign sequence on objects.

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: install.packages("ggplot2"). For the Hrate package this is different, since it comes from github. The devtools library needs to be installed, and then the install_github() function is used.

```
library(stringr)
library(entropy)
library(gsubfn)
```

```
## Loading required package: proto
```

```
#library(devtools)
#install_github("dimalik/Hrate")
library(Hrate)
```

Load files

Load the files with the cleaned and randomized sequences.

```
# signBase data
sign.data <- read.csv("~/Github/UpperPalCombinatorics/data/signBase/signBase_randomized.csv")
```

SignBase (Swabian Aurignacian) Estimations

Estimations of quantitative features for the Aurignacian objects and their sign codings as found in signBase.

```
# set counter
counter = 0
# initialize dataframe to append results to
estimations.df <- data.frame(object_id = character(0),
                             num.char = numeric(0), huni.chars = numeric (0),
                             hrate.chars = numeric(0),hrate.chars.random = numeric(0),
                             ttr.chars = numeric(0), rm.chars = numeric(0))

# start time
start_time <- Sys.time()
```

```r
for (i in 1:nrow(sign.data)){
  try({ # if the processing fails for a certain row, there will be no output for this row,
  # but the try() function allows the loop to keep running

  # collect info from row
  object_id <- sign.data$object_id[i]
  coding.sample <- as.character(sign.data$coding_clean[i])
  coding.sample.random <- as.character(sign.data$coding_random[i])
  # remove the white spaces around underscore
  coding.sample <- gsub(" _ ", "_", coding.sample, fixed = T)
  coding.sample.random <- gsub(" _ ", "_", coding.sample.random, fixed = T)

  # get number of characters
  num.char <- nchar(coding.sample)

  # split strings into UTF-8 characters
  chars <- unlist(strsplit(coding.sample, ""))
  chars.random <- unlist(strsplit(coding.sample.random, ""))
  # preprocess text with bag-of-words normalization
  # note: this is an important step! Without it, entropy rate estimation will fail
  # to yield realistic results due to issues with processing special characters.
  chars <- PreprocessText(chars)
  chars.random <- PreprocessText(chars.random)

  # unigram entropy estimation
  # calculate unigram entropy for characters (note that this is the same
  # for random ordering of chracter strings)
  chars.df <- as.data.frame(table(chars))
  # print(chars.df)
  huni.chars <- entropy(chars.df$Freq, method = "ML", unit = "log2")

  # entropy rate estimation
  # the values chosen for max.length and every.word will crucially
  # impact processing time. In case of "max.length = NULL" the length is n units
  # here use the try() function since entropy rate estimation can fail when particular
  # special characters are in the strings
  hrate.chars <- try(get.estimate(text = chars, every.word = 1, max.length = NULL))
  if (class(hrate.chars) == "numeric") {
    hrate.chars <- hrate.chars
  } else {
    hrate.chars = "NA"
  }
  # for random ordering of characters strings
  hrate.chars.random <- try(get.estimate(text = chars.random, every.word = 1, max.length = NULL))
  if (class(hrate.chars.random) == "numeric") {
    hrate.chars.random <- hrate.chars.random
  } else {
    hrate.chars.random = "NA"
  }

  # calculate type-token ratio (ttr)
  # note that this is the same for random ordering of strings
  ttr.chars <- nrow(chars.df)/sum(chars.df$Freq)
```

```
  # calculate repetition measure according to Sproat (2014)
  # note that this is the same for random ordering of strings
  # the maximum number of adjacent repetitions "R" possible is the sum of frequency counts minus 1.
  R <- sum(chars.df$Freq)-1
  # calculate the number of adjacent repetitions "r"
  r = 0
  if (length(chars) > 1){
    for (i in 1:(length(chars)-1)){
      if (chars[i] == chars[i+1]){
        r = r + 1
      } else {
        r = r + 0
      }
      }
    # calculate the repetition measure
    rm.chars <- r/R
  } else {
    rm.chars <- "NA"
  }

  # append results to dataframe
  local.df <- data.frame(object_id, num.char, huni.chars, hrate.chars,
                         hrate.chars.random, ttr.chars, rm.chars)
  estimations.df <- rbind(estimations.df, local.df)
}) # end of try()
# counter
counter <- counter + 1
# print(counter)
}
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 1.259485 secs
```

```
#estimations.df
```

Merge estimations with original file

```
sign.data.est <- merge(sign.data, estimations.df, by = "object_id")
```

Safe outputs to file.

```
write.csv(sign.data.est, "~/Github/UpperPalCombinatorics/results/signBase_features.csv", row.names = F)
```